

0.1 Kết luận

Trong phạm vi đề tài, đồ án đã xây dựng được một quy trình mô phỏng–hiển thị trong đó SUMO đảm nhiệm mô phỏng vi mô, Python điều khiển mô phỏng theo từng bước thông qua TraCI và Unity đảm nhiệm hiển thị không gian 3D theo thời gian thực. So với các cách quan sát kết quả mô phỏng truyền thống (thường tập trung vào 2D hoặc phụ thuộc nhiều vào thao tác dựng cảnh/đường đi thủ công), hệ thống của đồ án tập trung vào khả năng tự động hóa luồng dữ liệu và đồng bộ theo bước mô phỏng, qua đó hỗ trợ quan sát trực quan hơn trong các kịch bản có mật độ đối tượng cao hoặc khi cần thuyết minh kết quả.

Về kết quả đạt được, đồ án hoàn thành các nhóm chức năng chính sau:

- **Kết nối và đồng bộ thời gian thực:** triển khai kênh giao tiếp Unity–Python qua TCP/JSON và cơ chế tách khung bản tin bằng marker <END> để khắc phục đặc thù TCP hướng luồng; hỗ trợ bản tin dữ liệu và bản tin điều khiển (tạm dừng/tiếp tục/kết thúc).
- **Điều phối mô phỏng ổn định:** tổ chức đa luồng cho vòng lặp mô phỏng, các kênh nhận lệnh và (tuỳ chọn) kênh cập nhật từ Unity; bổ sung cơ chế dừng gọn để đảm bảo đóng kết nối và ghi kết quả đầy đủ.
- **Sinh dữ liệu bản đồ phục vụ benchmark:** xây dựng cách tạo mạng từ lưới ký tự theo hướng vết cạ (mỗi ô . là một node) để dễ mở rộng kích thước và đánh giá tải hệ thống; các thuật toán sinh mạng khác (ví dụ từ mê cung hoặc từ bản đồ thành phố) được giữ dưới dạng mã nguồn tham khảo trong thư mục SUMO_xml.
- **Chuẩn hoá dữ liệu trạng thái để hiển thị 3D:** trích xuất trạng thái xe/đèn giao thông/người đi bộ theo từng bước mô phỏng và chuyển đổi sang cấu trúc dữ liệu phù hợp để Unity dựng cảnh và cập nhật đối tượng.
- **Ghi log và tổng hợp thực nghiệm:** lưu thông tin di chuyển của xe theo định dạng bảng (CSV) và tạo thống kê tổng hợp theo lần chạy (JSON), phục vụ đối chiếu và đánh giá hiệu năng/độ ổn định.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế cần tiếp tục hoàn thiện. Thứ nhất, giao thức truyền thông hiện tại ưu tiên tính đơn giản (JSON) nên khi số lượng đối tượng lớn có thể phát sinh tải CPU/băng thông và làm giảm tốc độ cập nhật. Thứ hai, phần hiển thị trong Unity chủ yếu tập trung vào dựng và cập nhật trạng thái cơ bản; các yếu tố tăng tính chân thực (nội suy chuyển động, mô hình hành vi, hiệu ứng/điều khiển chi tiết theo làn) chưa được khai thác đầy đủ. Thứ ba, dù trong mã nguồn có nhiều hướng sinh bản đồ (mê cung, bản đồ thành

phổ), cấu hình chạy thực nghiệm hiện tại ưu tiên mạng vét cạn để benchmark, do đó tính “thực tế” về hình học mạng chưa phải trọng tâm.

Từ quá trình thực hiện, đề án rút ra một số kinh nghiệm chính: (i) trong hệ thống thời gian thực, cần thiết kế cơ chế phân định thông điệp rõ ràng khi dùng TCP để tránh lỗi khó truy vết; (ii) việc chuẩn hoá dữ liệu và thống nhất hệ toạ độ/đơn vị ngay từ đầu giúp giảm đáng kể công sức tích hợp giữa mô phỏng và hiển thị; và (iii) cơ chế dừng gọn và ghi log có ý nghĩa quan trọng để tăng tính lặp lại của thực nghiệm.

0.2 Hướng phát triển

Trong tương lai, đề án có thể được phát triển theo các hướng chính sau:

- **Tối ưu truyền thông và hiệu năng:** giảm chi phí serialize/parse bằng cách gom gói theo cấu trúc nhị phân, nén dữ liệu, hoặc chỉ gửi phần thay đổi theo bước; bổ sung cơ chế đo độ trễ đầu-cuối để đánh giá chất lượng đồng bộ.
- **Mở rộng nguồn sinh bản đồ:** đưa các thuật toán sinh mạng không vét cạn (ví dụ rút gọn node theo nút giao, hoặc sinh mạng theo mẫu thành phố) thành tùy chọn cấu hình chạy; bổ sung pipeline nhập mạng từ `OpenStreetMap/netconvert` để tăng khả năng ứng dụng thực tế.
- **Nâng cấp hiển thị 3D:** cải thiện nội suy chuyển động và điều khiển quay/đổi làn để tăng độ mượt; bổ sung cơ chế camera theo dõi, hiển thị chú giải và các lớp thông tin (mật độ, tốc độ trung bình) nhằm phục vụ phân tích.
- **Bổ sung kịch bản và đánh giá:** xây dựng bộ kịch bản benchmark chuẩn theo nhiều kích thước mạng và mật độ phương tiện; chuẩn hoá thước đo (FPS, độ trễ, tỉ lệ mất gói, thời gian di chuyển) và tự động hoá chạy thực nghiệm.
- **Cải thiện chất lượng phần mềm:** bổ sung kiểm thử cho các mô-đun sinh mạng, sinh route và đóng gói giao thức; chuẩn hoá cấu hình chạy và tài liệu hoá giao thức để thuận tiện bảo trì/mở rộng.